

Aplicação de Padrões de Projeto na API Karangos

Aluno: Vitor Satri
Curso: Análise e Desenvolvimento de Sistemas
Projeto-base: API Karangos desenvolvida em Programação de Scripts
Data: 02 de outubro de 2026

Resumo

O projeto Karangos já oferecia CRUDs de clientes, veículos e usuários por meio de uma API Java com Spring Boot, controllers REST, services, repositories JPA e PostgreSQL. Esta atividade acrescenta requisitos de reserva, venda, notificação, avaliação e composição de preço. Foram aplicados quatro padrões estudados: State, Observer, Adapter e Decorator.

As implementações mantêm os CRUDs existentes e isolam as novas regras em classes próprias. O resultado é um código no qual as transições de venda não ficam espalhadas em condicionais, novos avisos podem ser adicionados sem alterar o serviço de vendas, o contrato interno não depende da interface do avaliador legado e adicionais de preço podem ser combinados sem criar uma classe para cada combinação.

Situação inicial e requisitos acrescentados

A aplicação original separava as requisições HTTP, as regras de aplicação e a persistência nas classes de controller, service e repository. Os veículos já possuíam cliente associado, data de venda e preço de venda. Esses dados permitiram criar o fluxo de venda sem modificar o esquema do banco.

- Foram acrescentados os seguintes requisitos:
1. Um veículo disponível pode ser reservado para um cliente.
 2. Somente um veículo reservado pode ter a venda concluída.
 3. Uma reserva pode ser cancelada, mas uma venda concluída não pode voltar ao estado anterior.
 4. A conclusão de uma venda deve avisar todos os componentes interessados.
 5. A API deve consultar um avaliador legado sem expor seu formato ao restante da aplicação.
 6. Um orçamento pode receber taxa de veículo importado, tarifa de documentação e desconto de fidelidade.

Visão geral das escolhas

Padrão	Problema tratado	Principal ponto do código
State	Regras diferentes conforme o estágio da venda	<code>patterns/state/CarSaleStateContext.java</code>
Observer	Vários efeitos após uma venda	<code>patterns/observer/CarSalePublisher.java</code>
Adapter	Interface incompatível do avaliador legado	<code>patterns/adapter/LegacyValuationAdapter.java</code>
Decorator	Combinações variáveis de taxas e descontos	<code>patterns/decorator/SalePriceDecorator.java</code>

Padrão State

O que é e para que serve

State é um padrão comportamental que representa cada estado relevante por meio de um objeto próprio. O objeto de contexto delega o comportamento ao estado atual. O padrão é adequado quando uma operação permitida em um momento deve ser proibida ou executada de modo diferente em outro momento.

Sem esse padrão, o serviço de venda tenderia a acumular verificações como `if disponível`, `if reservado` e `if vendido` em vários métodos. À medida que novas transições fossem criadas, a mesma regra poderia ser repetida e ficar inconsistente.

Onde e quando foi usado

O fluxo do veículo possui três estados:

- `AVAILABLE` : não há cliente nem venda registrada;
- `RESERVED` : existe cliente, mas ainda não há data nem preço de venda;
- `SOLD` : existe data ou preço de venda.

O estado é calculado a partir dos campos que já existiam na entidade `car`. Dessa forma, não foi necessário criar uma coluna adicional apenas para a demonstração do padrão.

Como foi implementado

A interface `CarSaleState` define as operações de reservar, concluir e cancelar. Os estados concretos sobrescrevem somente as transições permitidas. O contexto escolhe a implementação correspondente aos dados do veículo.

```
public static CarSaleStateContext from(Car car) {
    if (car.getSellingDate() != null || car.getSellingPrice() != null) {
        return new CarSaleStateContext(car, new SoldCarState());
    }
    if (car.getCustomer() != null) {
        return new CarSaleStateContext(car, new ReservedCarState());
    }
    return new CarSaleStateContext(car, new AvailableCarState());
}
```

No estado reservado, a venda somente é concluída quando o preço é positivo:

```
public void complete(Car car, BigDecimal sellingPrice, LocalDate sellingDate) {
    if (sellingPrice == null || sellingPrice.signum() <= 0) {
        throw new IllegalArgumentException(
            "The selling price must be greater than zero.");
    }
    car.setSellingPrice(sellingPrice);
    car.setSellingDate(sellingDate);
}
```

Resultados alcançados

As transições válidas ficaram explícitas e concentradas nas classes de estado. Uma tentativa de vender um veículo disponível retorna conflito em vez de gravar dados incompletos. O teste `CarSaleStateContextTests` verifica a sequência disponível, reservado e vendido, além de confirmar que a venda direta de um veículo disponível é rejeitada.

Padrão Observer

O que é e para que serve

Observer é um padrão comportamental em que um publicador mantém uma coleção de observadores e os notifica quando ocorre um evento. Ele deve ser usado quando vários componentes precisam reagir ao mesmo fato e o produtor do evento não deve conhecer os detalhes de cada reação.

No projeto, a venda concluída exige pelo menos duas reações: registrar uma trilha de auditoria e preparar uma confirmação ao cliente. Essas reações não fazem parte da alteração de estado do veículo.

Onde e quando foi usado

`CarSaleService` cria um `CarSoldEvent` após salvar a venda. `CarSalePublisher` recebe todos os beans que implementam `CarSaleObserver` e encaminha o evento. Foram criados dois observadores:

- `SaleAuditObserver`, responsável pelo registro estruturado da operação;
- `CustomerNotificationObserver`, responsável por preparar a confirmação ao e-mail do cliente.

Como foi implementado

```
@Component
public class CarSalePublisher {
    private final List<CarSaleObserver> observers;

    public CarSalePublisher(List<CarSaleObserver> observers) {
        this.observers = List.copyOf(observers);
    }

    public void publish(CarSoldEvent event) {
        observers.forEach(observer -> observer.onCarSold(event));
    }
}
```

O serviço publica o evento sem depender das classes concretas:

```
publisher.publish(new CarSoldEvent(
    savedCar.getId(),
    savedCar.getPlates(),
    customer.getId(),
    customer.getEmail(),
    savedCar.getSellingPrice(),
    savedCar.getSellingDate()));
```

Resultados alcançados

O serviço de vendas permaneceu independente dos mecanismos de auditoria e comunicação. Um novo observador, como atualização de estoque ou geração de nota, pode ser incluído implementando a interface, sem alterar o publicador. O teste `CarSalePublisherTests` confirma que todos os observadores registrados recebem o mesmo evento.

Nesta versão acadêmica, os observadores registram as ações no log. O envio real de e-mail exigiria credenciais e um provedor externo, mas não mudaria o contrato do padrão.

Padrão Adapter

O que é e para que serve

Adapter é um padrão estrutural que converte a interface de um componente para o contrato esperado pela aplicação. Ele é útil ao integrar código legado, bibliotecas ou serviços externos que não podem ser alterados e trabalham com nomes, tipos ou formatos diferentes.

Onde e quando foi usado

A Karangos passou a oferecer avaliação estimada de um veículo. O componente legado recebe nome concatenado, ano, código de origem e devolve um valor inteiro em centavos. A aplicação trabalha com a entidade `Car` e com `BigDecimal`. Acoplar os services diretamente ao formato legado espalharia detalhes de conversão pelo código.

Como foi implementado

`VehicleValuationPort` é o contrato esperado pela aplicação. `LegacyValuationAdapter` traduz a entidade e converte centavos em valor monetário:

```
public BigDecimal estimate(Car car) {
    String vehicleName = car.getBrand() + " " + car.getModel();
    String originCode = Boolean.TRUE.equals(car.getImported()) ? "IMP" : "NAC";
    long valueInCents = legacyClient.calculateValueInCents(
        vehicleName, car.getYearManufacture(), originCode);

    return BigDecimal.valueOf(valueInCents, 2)
        .setScale(2, RoundingMode.HALF_UP);
}
```

`CarValuationService` depende apenas de `VehicleValuationPort`. Outro fornecedor poderá substituir o legado sem modificar o controller nem as regras que consomem a avaliação.

Resultados alcançados

Os detalhes da integração ficaram isolados. A API expõe um valor decimal consistente, enquanto o componente legado pode continuar usando centavos e códigos próprios. O teste `LegacyValuationAdapterTests` verifica a conversão e o formato com duas casas decimais.

O cliente legado deste anexo é uma simulação local e determinística. Em produção, a classe poderia encapsular uma chamada HTTP ou um SDK proprietário, mantendo o mesmo adapter.

Padrão Decorator

O que é e para que serve

Decorator é um padrão estrutural que envolve um objeto com outro objeto que implementa a mesma interface. Cada decorator acrescenta um comportamento e pode ser combinado com outros em tempo de execução. Ele é indicado quando existem recursos opcionais combináveis e a herança produziria muitas subclasses.

Onde e quando foi usado

O orçamento de venda parte de um preço-base e pode receber:

- taxa de 3% para veículo importado;
- tarifa fixa de R\$ 850,00 para documentação;
- desconto percentual de fidelidade.

Criar subclasses para todas as combinações geraria classes como preço importado com documentação, preço importado com desconto e outras variações. Os decoradores evitam essa multiplicação.

Como foi implementado

Todos os componentes implementam `SalePrice`. O serviço monta a cadeia conforme os dados do veículo e da requisição:

```

SalePrice price = new BaseSalePrice(basePrice);
if (Boolean.TRUE.equals(car.getImported())) {
    price = new ImportedVehicleFeeDecorator(price);
}
if (includeDocumentation) {
    price = new DocumentationFeeDecorator(price);
}
if (loyaltyDiscountPercent != null && loyaltyDiscountPercent.signum() > 0) {
    price = new LoyaltyDiscountDecorator(price, loyaltyDiscountPercent);
}

```

Cada decorador calcula seu valor a partir do componente anterior e também acrescenta uma descrição da composição.

Resultados alcançados

As opções podem ser combinadas sem mudar a classe-base. Novas regras, como garantia estendida ou taxa de entrega, podem ser implementadas em outro decorador. O teste `SalePriceDecoratorTests` combina taxa de importação, documentação e desconto e confirma o total de R\$ 93.465,00 para um preço-base de R\$ 100.000,00.

Integração com a API

As novas operações foram reunidas em `CarPatternController`, sem alterar as rotas dos CRUDs existentes.

Método e rota	Função	Padrão principal
GET /cars/{id}/sale/state	Consultar estágio da venda	State
POST /cars/{id}/sale/reserve	Reservar para um cliente	State
POST /cars/{id}/sale/complete	Concluir e emitir evento	State e Observer
POST /cars/{id}/sale/cancel	Cancelar a reserva	State
GET /cars/{id}/valuation	Consultar avaliação legada	Adapter
POST /cars/{id}/quote	Compor preço final	Decorator

O arquivo `exemplos-http.http` contém corpos de requisição prontos para o cliente HTTP do Visual Studio Code.

Organização do código

```

codigo-anexo/api/src/main/java/br/edu/fatecfranca/api/
|-- controllers/CarPatternController.java
|-- patterns/
|   |-- adapter/
|   |-- decorator/
|   |-- observer/
|   |-- state/
|   `-- web/
`-- services/
    |-- CarQuoteService.java
    |-- CarSaleService.java
    `-- CarValuationService.java

```

A entidade `Car` recebeu apenas o getter de `Customer`, necessário para que o fluxo de venda e o evento consultem o cliente associado. Não foi criada uma migration nova.

Verificação

Foi adicionada a dependência H2 somente para testes. O arquivo `src/test/resources/application.properties` desabilita o Flyway durante a suíte e cria um banco em memória. Assim, os testes não acessam nem alteram o PostgreSQL usado pela aplicação.

Comando executado:

```
.\mvnw.cmd test
```

Resultado em 02/10/2026:

- 28 testes executados;
- 0 falhas;
- 0 erros;
- compilação concluída com sucesso.

Cinco desses testes verificam diretamente State, Observer, Adapter e Decorator. Os demais testes originais dos controllers e da inicialização da aplicação também permaneceram aprovados.

Conclusão

Os quatro padrões atendem a problemas diferentes do mesmo domínio. State protege o ciclo de venda, Observer separa os efeitos posteriores, Adapter limita o acoplamento com o sistema legado e Decorator organiza as combinações de preço. As mudanças acrescentam requisitos observáveis à API e mantêm os CRUDs existentes.

Os padrões não foram inseridos apenas para criar novas classes. Cada um delimita uma parte sujeita a mudança: regras de transição, interessados em eventos, fornecedor de avaliação e composição comercial. Essa separação reduz alterações em cadeia quando o sistema evoluir.